TP 2 — Détection du sarcasme en arabe : déséquilibre, multi-tâche et généralisation

ArSarcasm-v2 · PLM zero-shot · embeddings statiques + réseaux profonds · PLM gelé et fine-tuné (mono- et multi-tâche) · tests de généralisation

Binôme ou individuel · Rendu : dépôt Git + compte-rendu (dernières pages)

1. Objectifs

- Construire un **prétraitement de l'arabe** correct, et vérifier qu'il ne **détruit pas les indices du sarcasme** (emojis, allongements, ponctuation, rires).
- Détecter le **sarcasme** (classes déséquilibrées) avec 5 familles d'approches : baseline TF-IDF, **PLM utilisé directement** (zero-shot), **embeddings statiques + réseaux profonds**, **PLM gelé** + tête profonde, **PLM fine-tuné**.
- Gérer le **déséquilibre** (pondération, ré-échantillonnage, seuil) et choisir la bonne métrique.
- Exploiter les autres annotations par **apprentissage multi-tâche** (sarcasme + sentiment) sur un encodeur partagé — lien direct avec le cours sur l'encodeur.
- Évaluer **comme en production** : test officiel jamais vu, décalage de prior, dialecte non vu, raccourcis (shortcuts), analyse d'erreurs.

2. Prérequis & environnement

- Cours 1 (embeddings), Cours 2 (attention), Cours 3 (bloc Transformer, encodeur, BERT).
- Python 3.10+ ; `torch`, `transformers`, `datasets`, `evaluate`, `scikit-learn`, `gensim`, `pyarabic`, `emoji` ; préprocesseur AraBERT (`pip install arabert`, ou `preprocess.py` du dépôt aub-mind/arabert).
- GPU : Colab ou Kaggle (une T4 suffit). Environnement géré avec uv (ou `requirements.txt` figé), dépôt Git créé dès la première séance.

3. Données — ArSarcasm-v2

Corpus de **15 548 tweets arabes** publié pour la tâche partagée WANLP 2021 (sarcasme et sentiment). Chaque tweet porte **trois annotations** : sentiment, sarcasme et dialecte. Le découpage train/test officiel est fourni et doit être respecté.

Élément	Détail
Source	github.com/iabufarha/ArSarcasm-v2 (dossier `ArSarcasm-v2/`)
Fichiers	`training_data.csv` (12 548 tweets) · `testing_data.csv` (3 000 tweets)
Colonnes	`tweet` · `sarcasm` (TRUE / FALSE) · `sentiment` (NEG / NEU / POS) · `dialect` (msa, egypt, gulf, levant, magreb)
Tâche du TP	Prédire la colonne `sarcasm` (binaire) à partir de `tweet`. `sentiment` et `dialect` servent de tâches auxiliaires (Partie E) et de sous-groupes d'évaluation — **jamais comme entrées du modèle**.
Métriques	**F1 de la classe sarcastique** (principale, métrique officielle — meilleur score publié : 0,6225) ; **PR-AUC** ; macro-F1. L'accuracy n'est **pas** une métrique acceptable ici (voir Q1).

Protocole imposé (identique pour tout le monde)

```
import pandas as pd
from sklearn.model_selection import train_test_split

BASE = "https://raw.githubusercontent.com/iabufarha/ArSarcasm-v2/main/ArSarcasm-v2/"
full = pd.read_csv(BASE + "training_data.csv")
test = pd.read_csv(BASE + "testing_data.csv")     # scellé jusqu'à la Partie F
```

```
full["id"], test["id"] = range(len(full)), range(len(test))

train, val = train_test_split(full, test_size=0.10, random_state=42,
                              stratify=full["sarcasm"])
```

Règles du jeu

• Toutes les décisions (prétraitement, architecture, hyperparamètres, nombre d'époques, **seuil de décision**) se prennent sur **val**. Le test officiel ne sert qu'aux modèles finaux, en Partie F.

• Toute transformation apprise (vocabulaire, TF-IDF, mise à l'échelle, poids de classes, ré-échantillonnage) est ajustée sur **train uniquement**.

• Chaque exécution est journalisée dans `results/results.csv` : modèle, prétraitement, graine, métriques val/test, temps d'entraînement, nombre de paramètres entraînés.

4. Prétraitement — à faire « comme il se doit »

Le prétraitement n'est pas une étape unique : **il dépend du modèle**. Un bon prétraitement pour TF-IDF peut dégrader un PLM, et inversement. Il doit être **audité, justifié, testé et mesuré**.

4.1 Audit du texte brut (avant toute normalisation)

- Affichez 30 tweets bruts de train et 30 de test. Vous trouverez **au moins un artefact de format** (introduit par l'export CSV, pas par l'auteur du tweet) : identifiez-le, mesurez sa fréquence dans train **et** dans test, et sa corrélation avec les labels.
- Comptez : URL, mentions @, hashtags, emojis, rires (هههه), ponctuation répétée, diacritiques, tatweel, lettres répétées, caractères latins, chiffres indiens (٠١٢).
- Doublons : exacts **et** après normalisation, dans train et entre train et test. Retirez de train les tweets présents dans test et rapportez leur nombre. Repérez les doublons aux **labels contradictoires** et décidez d'une politique (justifiée).

5. Déroulé

Partie A — Exploration, raccourcis & baseline

- Taux de sarcasme dans train et dans test, par dialecte et par sentiment.
- Classifieur trivial « jamais sarcastique » : accuracy et F1 sur val. Conclusion sur l'accuracy ?
- Baseline : TF-IDF (mots 1–2 + caractères 2–5) + régression logistique `class_weight="balanced"` ; **seuil de décision choisi sur val** (maximisation du F1 sarcastique).
- **Audit des raccourcis** : taux de sarcasme selon la présence d'une URL, d'une mention, de l'artefact de format (4.1). Évaluez un « classifieur à une seule règle » pour chacun : que vaut-il ?

Partie B — PLM utilisé directement (aucun entraînement)

- **B1. Zero-shot par NLI** : `MoritzLaurer/mDeBERTa-v3-base-xnli-multilingual-nli-2mil7`. Testez **au moins 3 formulations** d'hypothèse (arabe, français, anglais). Le score P(sarcastique) est **seuillé sur val** : même sans entraînement, la validation reste indispensable.
- **B2. Transfert par heuristique** : modèle de sentiment déjà fine-tuné `CAMeL-Lab/bert-base-arabic-camelbert-da-sentiment` + règle « sentiment négatif ⇒ sarcastique ». Inspectez `model.config.id2label`.

```
from transformers import pipeline
zs = pipeline("zero-shot-classification",
              model="MoritzLaurer/mDeBERTa-v3-base-xnli-multilingual-nli-2mil7")
labels = ["ساخرة",     # sarcastique
          "جادة"]      # sérieuse
template = "هذه التغريدة {}."   # « Ce tweet est {}. »
out = zs(tweet, candidate_labels=labels, hypothesis_template=template)

sa = pipeline("text-classification",
              model="CAMeL-Lab/bert-base-arabic-camelbert-da-sentiment")
print(sa.model.config.id2label)
```

Piège — contamination

• Le modèle de B2 a été fine-tuné sur ASTD, ArSAS et SemEval. Or ArSarcasm (v1), inclus dans ArSarcasm-v2, a été construit en **ré-annotant ASTD et SemEval 2017**.

• Il a donc peut-être déjà vu certains de vos tweets de test. Gardez-le en tête en interprétant B2.

Partie C — Embeddings statiques + réseaux profonds (PyTorch)

- Tokenisation en mots sur le texte normalisé ; vocabulaire construit sur train (fréquence ≥ 2) ; **taux d'OOV** sur val et test.
- Couche d'embedding : (i) aléatoire ; (ii) **AraVec** (Twitter, unigrammes, 100 ou 300 dim.), figée puis dégelée. Appliquez `clean_str` d'AraVec avant la recherche dans son vocabulaire.
- Architectures — même budget d'entraînement, early stopping sur val F1-sarcastique :
  - **C1.** BiLSTM (dernier état caché ou max-pooling).
  - **C2.** BiLSTM + **pooling attentionnel** implémenté par vous : $\alpha_i = \text{softmax}(v^T \tanh(W h_i))$, $c = \Sigma\, \alpha_i\, h_i$.
  - **C3. Mini-encodeur Transformer** : 2 couches, d_model = 128, 4 têtes, encodage positionnel sinusoïdal, masque de padding. Ablation : **sans** encodage positionnel.
  - **C4.** Optionnel **CNN sur caractères** (char-CNN) : capte-t-il mieux allongements, rires et emojis ?
- **Déséquilibre** — sur le meilleur modèle de C1–C3, comparez : (a) perte pondérée ; (b) sur-échantillonnage de la classe minoritaire (**dans train uniquement**) ; (c) perte standard + seuil optimisé sur val. Optionnel : focal loss.

Partie D — PLM gelé : l'encodeur comme extracteur de représentations

- **D1.** `aubmindlab/bert-base-arabertv02-twitter` gelé : [CLS] vs **moyenne des tokens** (pondérée par le masque d'attention) → régression logistique (seuil sur val).
- **D2.** États de **tous les tokens** (encodeur gelé) → tête BiLSTM + attention de C2. **Visualisez l'attention sur 5 tweets sarcastiques.**

Partie E — Fine-tuning : mono-tâche vs multi-tâche

- Deux PLM pré-entraînés sur des tweets : `aubmindlab/bert-base-arabertv02-twitter` et `UBC-NLP/MARBERTv2`.
- **E1. Mono-tâche** (sarcasme) : perte pondérée, seuil choisi sur val.
- **E2. Multi-tâche** : encodeur partagé + 2 têtes (sarcasme, sentiment) ; perte $L = L_{sarc} + \lambda \cdot L_{sent}$, avec $\lambda \in \{0{,}3 ; 0{,}5 ; 1\}$ choisi sur val. Optionnel : 3e tête « dialecte ».
- Point de départ : lr 2e-5, batch 32, 3 à 5 époques, warmup 10 %, weight decay 0,01, fp16, early stopping sur val F1-sarcastique. **3 graines** par modèle final (moyenne ± écart-type).

```
class MultiTask(nn.Module):
    def __init__(self, name):
        super().__init__()
        self.enc = AutoModel.from_pretrained(name)
        h = self.enc.config.hidden_size
        self.sarc, self.sent = nn.Linear(h, 2), nn.Linear(h, 3)

    def forward(self, input_ids, attention_mask):
        out = self.enc(input_ids=input_ids, attention_mask=attention_mask)
        cls = out.last_hidden_state[:, 0]
        return self.sarc(cls), self.sent(cls)

# loss = ce_sarc(logits_sarc, y_sarc) + lam * ce_sent(logits_sent, y_sent)
```

Partie F — Tests de généralisation (le test officiel est « descellé »)

Appliquez F1 à F6 à la baseline et aux meilleurs modèles de C, D, E1 et E2.

- **F1. Décalage de prior** : comparez le taux de sarcasme de val et de test. Avec le seuil choisi sur val, quel F1 sur le test ? **A posteriori seulement** (analyse, pas sélection) : tracez F1 = f(seuil) sur le test.
- **F2. Dialecte non vu** : retirez tous les tweets egypt de train, réentraînez votre meilleur modèle E, évaluez sur les tweets egypt du test ; comparez au même modèle entraîné sur tout train.
- **F3. Raccourcis** : scores sur les sous-ensembles du test avec / sans URL. Réentraînez en supprimant les jetons URL et mention : qu'est-ce qui change ?

- **F4. Indices du sarcasme** : sur les tweets sarcastiques correctement détectés, supprimez emojis, rires et ponctuation répétée ; mesurez la **chute du taux de détection**.
- **F5. Significativité** : 3 graines, IC à 95 % par bootstrap (1 000 rééchantillonnages), test de McNemar entre vos deux meilleurs modèles.
- **F6. Analyse d'erreurs** : 30 erreurs (15 faux négatifs, 15 faux positifs) classées en catégories — connaissance du monde / contexte requis, ironie sans marqueur, marqueur mal interprété, dialecte, **bruit d'annotation**. Croisez avec les doublons aux labels contradictoires trouvés en 4.1.

Partie G — Comparaison & décision

- Tableau final : qualité, robustesse, temps d'entraînement, **latence d'inférence par tweet sur CPU**, taille du modèle, test de généralisation et performances.
- Matrice de confusion du modèle retenu (au seuil retenu).

6. Questions (à traiter dans le compte-rendu)

- **Q1.** Taux de sarcasme dans train, val et test, globalement et par dialecte. Accuracy et F1 du classifieur « jamais sarcastique ». Quelle métrique retenez-vous, et pourquoi ?
- **Q2.** Raccourcis : quels indices superficiels (artefact de format, URL, mentions) sont corrélés au label ? Que vaut un classifieur à une seule règle ? Qu'en concluez-vous sur la validité de vos scores ?
- **Q3.** Quels choix de prétraitement avez-vous faits par famille de modèles ? Lesquels détruisent des indices du sarcasme (ablation 4.3, chiffres à l'appui) ?
- **Q4.** Zero-shot (B1) : score obtenu et sensibilité à la formulation de l'hypothèse (arabe / français / anglais). Pourquoi un modèle NLI peut-il classer sans exemple d'entraînement ? Quel rôle joue l'attention entre prémisse et hypothèse ?
- **Q5.** Heuristique « négatif ⇒ sarcastique » (B2) : précision, rappel, F1. Que révèle-t-elle de la relation entre sarcasme et sentiment dans ce corpus ? Son score est-il exempt de contamination ?
- **Q6.** Déséquilibre : comparez pondération de la perte, sur-échantillonnage et optimisation du seuil (tableau). Pourquoi le sur-échantillonnage doit-il être fait après la séparation train / val ?
- **Q7.** C1 vs C2 vs C3 (et C4) : résultats. Les poids d'attention se concentrent-ils sur les marqueurs attendus (emojis, rires, allongements, ponctuation) ? Effet de l'encodage positionnel ?
- **Q8.** PLM gelé (D) vs fine-tuné (E1) : gain et coût (temps, paramètres entraînés). AraBERT-twitter vs MARBERT : lequel, et pourquoi ?
- **Q9.** Multi-tâche (E2) : l'ajout du sentiment améliore-t-il la détection du sarcasme ? Effet de λ. Pourquoi une tâche auxiliaire peut-elle aider… ou nuire ?
- **Q10.** Décalage de prior (F1) : écart entre le seuil choisi sur val et le seuil optimal a posteriori sur le test. Que mettriez-vous en place en production (recalibration, monitoring) ?
- **Q11.** Dialecte non vu (F2) : chute mesurée. Combien de tweets maghrébins (magreb) dans train et dans test ? Comment évalueriez-vous honnêtement ce modèle avant un déploiement sur des tweets marocains ?
- **Q12.** Analyse d'erreurs (F6) : catégories dominantes, part du bruit d'annotation. Quel modèle mettriez-vous en production, et pourquoi (qualité, robustesse, coût, latence — chiffres à l'appui) ?

7. Livrables & reproductibilité

- **Dépôt Git** : `README.md` (commandes d'exécution), `pyproject.toml` + `uv.lock` (ou `requirements.txt` figé), `src/` (data, preprocess, models, train, evaluate), `notebooks/`, `tests/`, `results/results.csv`, `predictions/`.
- Graines fixées (random, numpy, torch, transformers) ; split imposé ; aucune donnée de test dans train.
- **Prédictions du test** de chaque modèle final : `predictions/<modèle>_seed<k>.csv` (id, y_true, y_pred, probabilités) — l'enseignant recalcule les scores.
- Le compte-rendu ci-après, rempli (6 pages maximum hors annexes).

COMPTE-RENDU — TP 2

À remplir et à rendre (avec le lien du dépôt)

Identité

Binôme ou individuel : _____ / _____

Filière / groupe : _____ Date : _____ Lien du dépôt : _____

PLM fine-tunés : _____ Prétraitement PLM utilisé : _____

Courbe F1 = f(seuil) sur le test (F1) — coller la figure

Attention sur 5 tweets sarcastiques (D2) — coller la figure

Réponses aux questions

Q1. Taux de sarcasme dans train, val et test, globalement et par dialecte. Accuracy et F1 du classifieur « jamais sarcastique ». Quelle métrique retenez-vous, et pourquoi ?

Q2. Raccourcis : quels indices superficiels (artefact de format, URL, mentions) sont corrélés au label ? Que vaut un classifieur à une seule règle ? Qu'en concluez-vous sur la validité de vos scores ?

Q3. Quels choix de prétraitement avez-vous faits par famille de modèles ? Lesquels détruisent des indices du sarcasme (ablation 4.3, chiffres à l'appui) ?

Q4. Zero-shot (B1) : score obtenu et sensibilité à la formulation de l'hypothèse (arabe / français / anglais). Pourquoi un modèle NLI peut-il classer sans exemple d'entraînement ? Quel rôle joue l'attention entre prémisse et hypothèse ?

Q5. Heuristique « négatif ⇒ sarcastique » (B2) : précision, rappel, F1. Que révèle-t-elle de la relation entre sarcasme et sentiment dans ce corpus ? Son score est-il exempt de contamination ?

Q6. Déséquilibre : comparez pondération de la perte, sur-échantillonnage et optimisation du seuil (tableau). Pourquoi le sur-échantillonnage doit-il être fait après la séparation train / val ?

Q7. C1 vs C2 vs C3 (et C4) : résultats. Les poids d'attention se concentrent-ils sur les marqueurs attendus (emojis, rires, allongements, ponctuation) ? Effet de l'encodage positionnel ?

Q8. PLM gelé (D) vs fine-tuné (E1) : gain et coût (temps, paramètres entraînés). AraBERT-twitter vs MARBERT : lequel, et pourquoi ?

Q9. Multi-tâche (E2) : l'ajout du sentiment améliore-t-il la détection du sarcasme ? Effet de λ. Pourquoi une tâche auxiliaire peut-elle aider… ou nuire ?

Q10. Décalage de prior (F1) : écart entre le seuil choisi sur val et le seuil optimal a posteriori sur le test. Que mettriez-vous en place en production (recalibration, monitoring) ?

Q11. Dialecte non vu (F2) : chute mesurée. Combien de tweets maghrébins (magreb) dans train et dans test ? Comment évalueriez-vous honnêtement ce modèle avant un déploiement sur des tweets marocains ?

Q12. Analyse d'erreurs (F6) : catégories dominantes, part du bruit d'annotation. Quel modèle mettriez-vous en production, et pourquoi (qualité, robustesse, coût, latence — chiffres à l'appui) ?

Décision & justification

Modèle retenu pour la production, seuil de décision et pourquoi (qualité / robustesse / coût / latence, chiffres à l'appui) :

Limites & pistes d'amélioration